

Audio NLP: Processing, Evaluation & Review

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology



LMH

Lady Margaret Hall

June 12, 2026



Car Horn (0.05)

Dog Barking (0.81)

Drill (0.03)

1. Introduction & Motivation
2. Learning Outcomes
3. Fundamentals of Sound
4. Pre-processing Pipeline
5. STFT & Spectrograms
 1. Fourier Transforms
 2. Short-Time Fourier Transform (STFT)
 3. Spectrograms
6. Mel-Scale & Filterbanks
 1. Perceptual Frequency Scales
 2. Mel Scale
 3. Mel Filterbanks

4. Choosing Features
5. Mel-Spectrograms
6. Log-Mel Features
7. MFCC Computation
7. Automatic Speech Recognition (ASR)
8. Spoken Language Understanding (SLU)
9. Text-to-Speech (TTS) & Vocoder
10. Evaluation Metrics
11. LLM Judges
12. Challenges & Current Research
13. Summary
14. References

Speech is the **most natural form of communication**.

What: Where speech interfaces are essential.

- ▶ Voice assistants (e.g., Amazon Echo)
- ▶ Accessibility: captions, screen readers
- ▶ Hands-free operation (cars, IoT)
- ▶ Multilingual interaction translation

Why: Speech enables more intuitive and inclusive interaction.

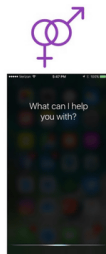
Why Audio in NLP? (cont.)



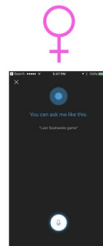
AMAZON'S ALEXA



GOOGLE'S ASSISTANT



APPLE'S SIRI



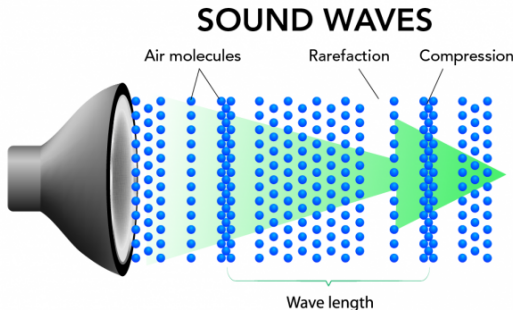
MICROSOFT'S CORTANA

- ▶ Audio data is abundant and diverse, encompassing speech, music, and environmental sounds.
- ▶ Traditional NLP methods focus on text, but audio data requires specialized techniques for processing.
- ▶ Many NLP systems now start from speech, not just text (e.g., live captions, real-time translation).
- ▶ The rise of voice assistants and audio-based applications highlights the need for effective audio processing in NLP.
- ▶ Audio signals contain rich information that can enhance understanding and interaction in various applications.

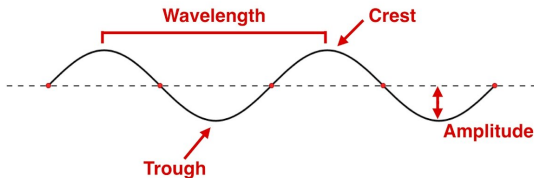
By the end of this session, you will be able to:

- ▶ Understand the fundamentals of audio processing in NLP.
- ▶ Explain how raw audio becomes features (MFCC, Mel/Log-Mel).
- ▶ Read reason about spectrograms (STFT, mel scale).
- ▶ Explore various architectures for Automatic Speech Recognition (ASR).
- ▶ Implement Text-to-Speech (TTS) systems using modern techniques.
- ▶ Connect features $\rightarrow$ ASR/NLU/TTS.
- ▶ Choose/evaluate features with simple metrics.
- ▶ Recognize the challenges and future directions in audio processing for NLP.

Fundamentals of Sound



- ▶ Sound is a type of energy produced by vibrating objects.
- ▶ It travels through air (or other media) as waves.
- ▶ Sound waves are longitudinal waves, meaning the particle displacement is parallel to the direction of wave propagation.



- ▶ **Waveform** $x(t)$: The shape of the sound wave, representing how air pressure changes over time.
- ▶ **Frequency**: Number of cycles (oscillations) per second, measured in Hertz (Hz). Determines the pitch of the sound.
- ▶ **Amplitude**: The height of the wave, representing the loudness or intensity of the sound.
- ▶ **Pitch/Formants**: The perceived frequency of a sound and its resonant frequencies, respectively.



To convert an analog signal into a digital one, two main steps are used:

- ▶ **Sampling:** The process of taking measurements of the continuous signal at regular time intervals.
- ▶ **Quantization:** The process of rounding off the amplitude of each sample to the nearest value on a finite scale.

The result is a **discrete signal**, denoted as $x[n]$, which is a sequence of numbers representing the original analog signal.



Why do we digitize signals?

- ▶ **Computer Processing:** Computers can only understand and manipulate discrete numerical values.
- ▶ **Reproducibility:** Digital signals can be copied and transmitted without any loss of quality.
- ▶ **Processability:** Digital signals are easier to process and manipulate using various algorithms for tasks such as noise reduction, filtering, and compression.

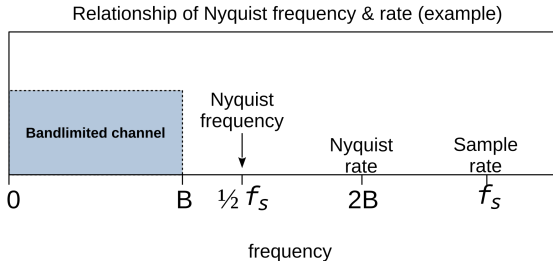
- ▶ **Sampling Rate** (f_s): The number of samples taken per second, measured in Hertz (Hz) or kilohertz (kHz). Common values include 16 kHz for speech and 44.1 kHz for music.
- ▶ **Nyquist–Shannon Sampling Theorem**: States that to perfectly reconstruct a continuous signal, the sampling rate (f_s) must be at least twice the maximum frequency ($f_{\max}$) of the signal.

$$f_s \geq 2f_{\max}$$

Failing to meet this condition can cause **aliasing**, where high frequencies are incorrectly represented as lower frequencies. The Nyquist limit, also known as the Nyquist frequency, is defined as half of the sampling rate:

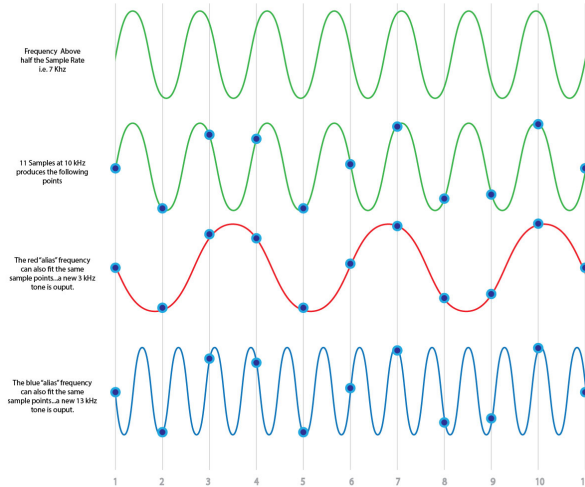
$$f_{\text{Nyq}} = \frac{f_s}{2}$$

- **Quantization:** The process of mapping continuous amplitude values to a finite set of discrete levels. This process introduces **quantization noise**, which is a form of error.



- **Aliasing:** If the sampling rate is too low, higher frequencies are misrepresented as lower frequencies, causing distortion.

Aliasing



- ▶ **Bit Depth:** Common depths include 8-bit (256 levels), 16-bit (65,536 levels), and 24-bit (16.7 million levels). Higher bit depth allows for more precise representation of amplitude.
- ▶ **Bitrate:** The amount of data processed per second, calculated as:

$$\text{Bitrate} = \text{Sampling Rate} \times \text{Bit Depth} \times \text{Number of Channels}$$

For example, a stereo audio file with a sampling rate of 44.1kHz and a bit depth of 16-bit has a bitrate of:

$$44,100 \times 16 \times 2 = 1,411,200 \text{ bits per second} = 1.4112 \text{ Mbps}$$

Pre-processing Pipeline

1. Pre-emphasis Filter

- ▶ Enhances high-frequency components of the audio signal.
- ▶ Equation:

$$y[n] = x[n] - \alpha x[n - 1]$$

where $x[n]$ is the input signal, $y[n]$ is the output, and α is typically between 0.95 and 0.97.

- ▶ Helps balance the frequency spectrum and improves feature extraction.

2. Framing

- ▶ Audio signals are divided into short frames (e.g., 25 ms) to capture local temporal features.
- ▶ Each frame overlaps with the next (commonly 10 ms overlap).

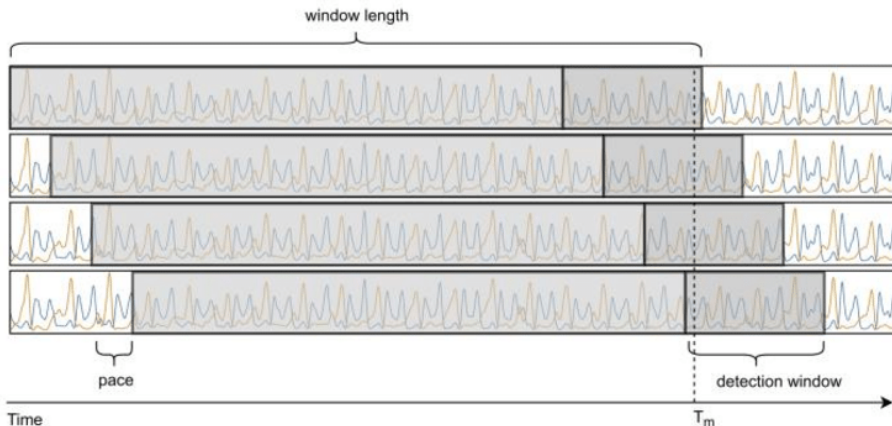
3. Windowing

- ▶ Each frame is multiplied by a window function (commonly Hamming window) to reduce spectral leakage.
- ▶ Hamming window equation:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

where N is the frame length.

Pre-processing Pipeline Details (cont.)



Waveform segmented into overlapping frames and windowed (source: ResearchGate)

Summary of Pre-processing Steps

1. **Pre-emphasis:** Boosts high frequencies.
2. **Framing:** Splits signal into short, overlapping segments.
3. **Windowing:** Applies Hamming window to each frame.



STFT & Spectrograms

- **Discrete Fourier Transform (DFT):** A mathematical operation that converts a signal from the time domain to the frequency domain. It transforms N samples of a discrete signal, $x[n]$, into N frequency bins, $X[k]$. The formula for the DFT is:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

where $x[n]$ is the signal in the time domain, $X[k]$ is the signal in the frequency domain, N is the total number of samples, n is the sample index, and k is the frequency bin index.

- **Fast Fourier Transform (FFT):** This is not a different transform, but rather a computationally efficient algorithm for calculating the DFT.

To apply the Fourier transform in practice, you typically:

1. **Choose a window length (N):** This determines the number of samples to be transformed at a time.
2. **Choose a hop size:** This is the number of samples to advance before taking the next window.

This process involves a fundamental trade-off: a longer window (N) provides better **frequency resolution** (ability to distinguish between close frequencies) but worse **time resolution** (ability to pinpoint when an event occurred).

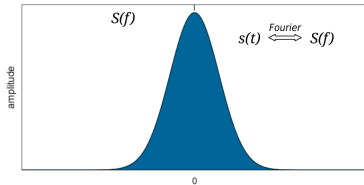
Conversely, a shorter window improves time resolution at the expense of frequency resolution.

We use Fourier transforms to analyze the frequency content of signals. The resulting frequency information (spectra) is essential for:

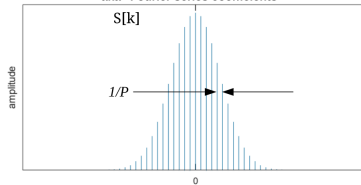
- ▶ **Building Spectrograms:** Visual representations of how the frequency content of a signal changes over time.
- ▶ **Extracting Features:** Deriving key characteristics from a signal that can be used for tasks like speech recognition, music information retrieval, and audio classification.

Fourier Transforms You'll Actually Use (cont.)

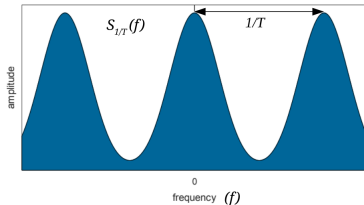
Fourier transform of a function $s(t)$ (which is not shown)



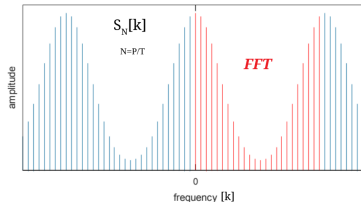
Transform of the periodic summation of $s(t)$
aka "Fourier series coefficients"



Transform of periodically sampled $s(t)$
aka "Discrete-time Fourier transform"



Transform of both periodic sampling and periodic summation
aka "Discrete Fourier transform"



Depiction of a Fourier transform. Source: Wikipedia

- ▶ STFT is a method to analyze the frequency content of a signal over time.
- ▶ It involves applying the Fourier Transform to short overlapping segments (frames) of the signal.
- ▶ The STFT provides a time-frequency representation of the signal.

► What:

- Slide a window across the signal.
- Compute a DFT per frame.

► Continuous form:

$$X(t, \omega) = \int x(\tau) w(\tau - t) e^{-j\omega\tau} d\tau$$

► Discrete idea:

- DFT of windowed frame at each time step.
- Sources: Wikipedia, courses.physics.illinois.edu

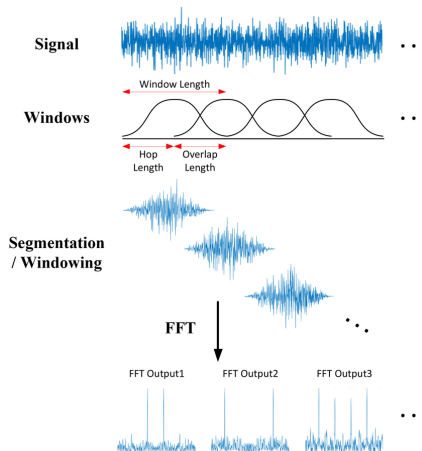
► How:

- Choose window: Hann / Hamming
- Frame length: e.g., 25 ms
- Hop: e.g., 10 ms

► Why:

- Speech is quasi-stationary in short frames.
- STFT captures how spectrum changes over time.

Short-Time Fourier Transform (STFT) (cont.)



STFT process: Signal divided into frames, each transformed into frequency domain.

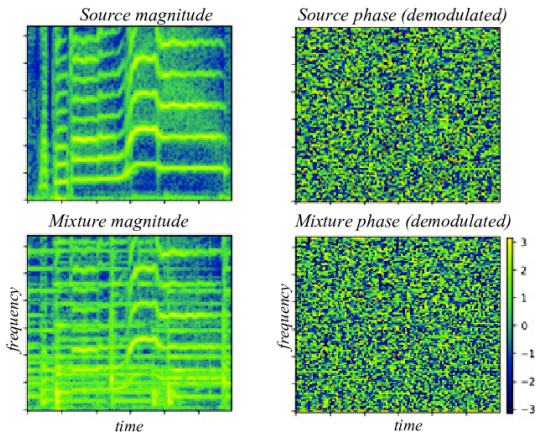
- The STFT is computed as:

$$STFT(x[n]) = \sum_{m=-\infty}^{\infty} x[m]w[n-m]e^{-j2\pi fm}$$

where $w[n-m]$ is a window function applied to each frame.

- ▶ The result is a complex-valued matrix, where each column represents the frequency content of a frame.
- ▶ The magnitude of the STFT gives the amplitude of each frequency at each time step.
- ▶ The phase of the STFT provides information about the timing of the frequency components.

Short-Time Fourier Transform (STFT) (cont.)



Magnitude and phase of STFT: Magnitude shows amplitude, phase shows timing of frequencies.

- ▶ A spectrogram is a visual representation of the STFT.
- ▶ It displays time on the x-axis, frequency on the y-axis, and color intensity represents amplitude.
- ▶ Spectrograms are widely used in audio analysis, speech recognition, and music processing.

► What:

- Heatmap of energy vs. time (x-axis) and frequency (y-axis).
- Color represents magnitude (often in dB).

► How:

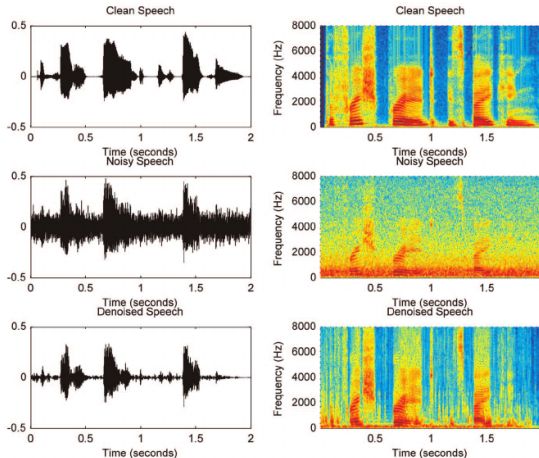
- Magnitude of STFT per time–frequency bin.
- Often log-scaled:

$$20 \log_{10} |X|$$

► Why:

- Visual patterns (formant tracks, harmonics, fricatives) → model-friendly features.
- Example: vowel formants appear as bright horizontal bands.

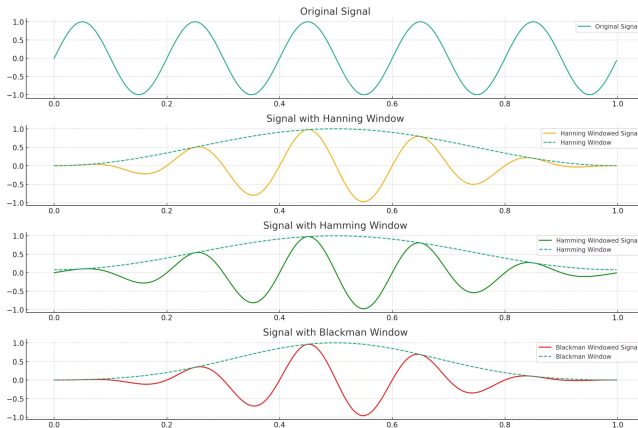
Spectrograms (cont.)



Example of a spectrogram: Time vs Frequency representation of an audio signal.

- ▶ Spectrograms can be computed using various window functions (e.g., Hamming, Hanning) and different frame sizes.
- ▶ They provide insights into how the frequency content of a signal changes over time.

Spectrograms (cont.)



Spectrogram variations: Different window functions and frame sizes affect the appearance of the spectrogram.



Mel-Scale & Filterbanks

► What:

- Human pitch perception is **non-linear**.
- We perceive changes at low frequencies more finely than at high frequencies.
- Example: Difference between 200 Hz and 300 Hz feels larger than between 3000 Hz and 3100 Hz.

► How:

- Map raw frequency in Hz onto perceptual scales:
 - **Mel scale** – approximates human pitch perception.
 - **Bark scale** – based on critical bands of hearing.
 - **ERB scale** – Equivalent Rectangular Bandwidth, linked to auditory filter widths.
- Each scale compresses high-frequency regions while preserving low-frequency resolution.

► Why:

- Features reflecting human hearing are more effective for **Automatic Speech Recognition (ASR)** and **Text-to-Speech (TTS)**.
- Speech features (e.g., MFCCs) rely heavily on perceptual scales.

► Focus: Mel Scale

- Most widely used in speech/audio tasks.
- Roughly linear spacing below 1 kHz, logarithmic above.
- Wikipedia reference: [Mel scale page](#).

► What:

- Mel scale converts frequency f (Hz) into perceptual pitch units (mels).
- Forward transform:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

- Inverse transform:

$$f = 700 \left(10^{\frac{m}{2595}} - 1 \right)$$

- Source: Wikipedia.

► How:

- Use Mel spacing to set filterbank center frequencies.
- Commonly used in **Mel Filterbanks** and **MFCCs**.

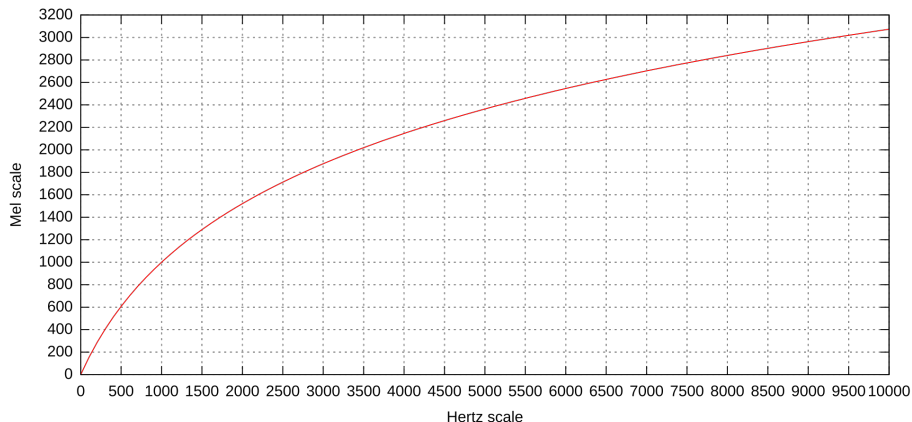
► Why:

- Aligns frequency resolution with **human pitch perception**.
- Compresses high-frequency regions (less perceptually important).
- Reduces unhelpful detail while retaining speech-relevant cues.

► Example:

- By convention, $1000 \text{ Hz} \approx 1000 \text{ mels}$.
- Below 1 kHz: roughly linear relation.
- Above 1 kHz: increasingly logarithmic.

Mel Scale: Definition & Intuition (cont.)

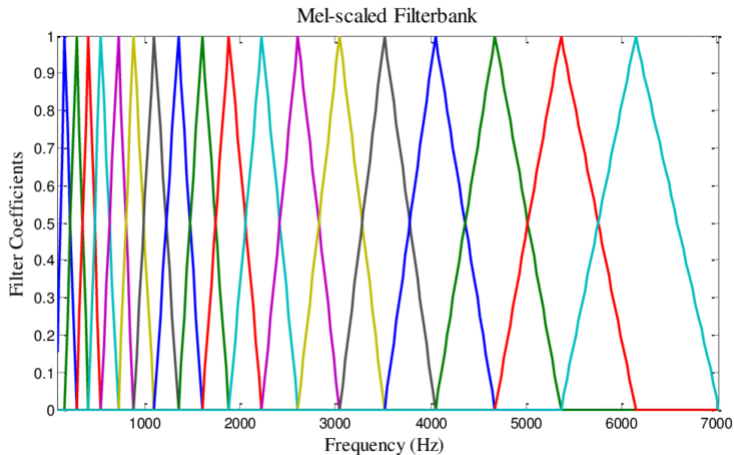


Plots of pitch mel scale versus hertz scale. Source: Wikipedia

Mel Filterbanks:

- ▶ The frequency axis is warped to the Mel scale.
- ▶ Typically, N triangular filters (e.g., $N = 40$) are created, spaced evenly in the Mel domain.
- ▶ Each filter collects energy from a range of frequencies, emphasizing perceptually important bands.
- ▶ The output is a set of Mel filterbank energies, which are used as features for audio and speech processing tasks.

Mel Filterbanks (cont.)



Filterbank Construction Steps:

1. Convert the lower and upper frequency bounds to Mel scale.
2. Linearly space $N + 2$ points between these bounds in Mel scale.
3. Convert these points back to Hz to get filter edges.
4. For each filter k , define a triangular response:

$$H_k(f) = \begin{cases} 0, & f < f_{k-1} \\ \frac{f - f_{k-1}}{f_k - f_{k-1}}, & f_{k-1} \leq f < f_k \\ \frac{f_{k+1} - f}{f_{k+1} - f_k}, & f_k \leq f < f_{k+1} \\ 0, & f \geq f_{k+1} \end{cases} \quad (1)$$

5. Multiply the power spectrum by each filter and sum to get filterbank energies.



- ▶ Different feature types capture sound information at different levels:
 - **Spectrogram (linear-Hz):** closest to the raw physics of sound; preserves fine high-frequency detail.
 - **Mel-spectrogram:** compresses the high-frequency range using perceptual spacing, better reflecting human hearing.
 - **Log-Mel:** adds logarithmic compression to mimic loudness perception and control dynamic range.
 - **MFCCs (Mel-Frequency Cepstral Coefficients):** apply a DCT to Log-Mel features, creating compact, decorrelated coefficients widely used in classic ASR.

► Common processing pipeline:

Waveform $\rightarrow$ STFT $\rightarrow$ Mel scaling (optional) $\rightarrow$ log $\rightarrow$ DCT (optional)

► Why this matters:

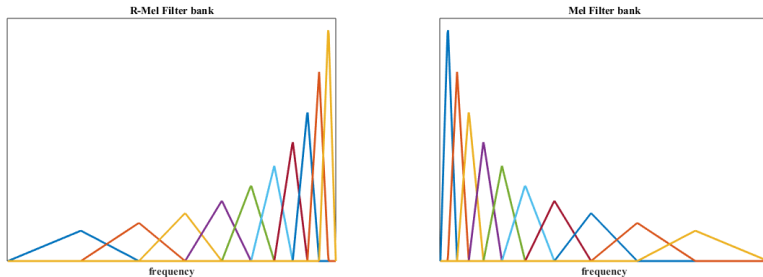
- More compact and stable features are easier for machine learning models to work with.
- Traditional models (like GMM-HMMs, SVMs) rely heavily on these, but even neural networks benefit when input dimensionality is reduced.

- ▶ A mel-spectrogram is a **time** $\times$ **mel-band matrix** created by re-binning STFT magnitudes through triangular filters on the Mel scale.
- ▶ The process:
 - Start with STFT $\rightarrow$ compute the power spectrum.
 - Apply a mel filterbank (typically 40–80 bands).
 - Take logarithms of the magnitudes to compress the dynamic range.
- ▶ Intuition:
 - Compact representation, aligned with human pitch perception.
 - Retains enough structure to capture speech/music content.
 - Works especially well with **CNNs** and **TTS acoustic models**.

- Equation (conceptual form):

$$E_m = \sum_k |X[k]|^2 H_m[k]$$

where H_m is the m -th triangular mel filter applied to the power spectrum.



Mel filter bank. Source: Wikipedia

- ▶ Log-Mel features are created by taking the logarithm of mel-band energies:

$$\log(E_m)$$

where E_m is the energy in the m -th mel band.

- ▶ Processing pipeline:

- Compute STFT $\rightarrow$ apply mel filterbank $\rightarrow$ mel-band energies.
- Apply logarithmic compression, either:

$$\log(E_m) \quad \text{or} \quad \log(1 + \alpha E_m)$$

to avoid issues with very small values.

► Why this step matters:

- Human perception of loudness is approximately logarithmic.
- Log scaling stabilizes feature magnitudes, preventing loud frames from dominating.
- Improves optimization for downstream machine learning models.

► Applications:

- Ubiquitous in modern **TTS systems** (e.g., WaveNet-style neural vocoders).
- Widely used in speech recognition pipelines.
- References: arXiv, Tacotron 2 project.

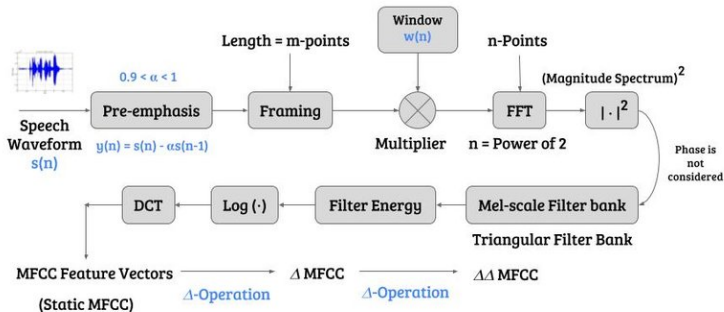
- ▶ Mel-Frequency Cepstral Coefficients (MFCCs) provide a compact summary of the log-Mel spectrum, obtained by decorrelating features with a Discrete Cosine Transform (DCT).
- ▶ This representation captures the **spectral envelope** (linked to vocal-tract shape) while discarding fine pitch details.
- ▶ Typical processing pipeline:
 - (Optional) pre-emphasis to boost high frequencies.
 - Frame the signal, apply a window, and compute the STFT.
 - Apply a mel filterbank to obtain band energies.
 - Take logarithms of the energies.
 - Apply DCT to decorrelate; keep the first Q coefficients (e.g., 13).
 - Optionally append temporal derivatives (Δ) to capture dynamics.

► Why MFCCs are important:

- Historically very strong for HMM-GMM based ASR systems.
- Emphasize formant structure and vocal-tract characteristics.
- Tend to suppress pitch information, which is often less useful for recognition tasks.

- ▶ MFCCs are widely used in speech and audio processing.
- ▶ They capture the timbral texture of audio signals.
- ▶ Typically, only the first 13 coefficients are used for feature extraction.

MFCC Computation (cont.)



A block diagram for MFCC computation: From audio signal to MFCC features.

Pipeline:

1. Compute log-Mel spectrogram
2. Apply Discrete Cosine Transform (DCT)
3. Keep first 13 coefficients (MFCCs)

MFCC Equation:

$$c_k = \sum_{n=1}^N \log(E_n) \cos \left[\frac{\pi k}{N} \left(n - \frac{1}{2} \right) \right] \quad (2)$$

where:

- ▶ c_k is the k -th MFCC coefficient
- ▶ E_n is the energy in the n -th Mel filter
- ▶ N is the total number of Mel filters

- ▶ MFCCs are built from three main mathematical steps:

1. **Band energies (Mel filtering):**

$$E_m = \sum_k |X[k]|^2 H_m[k]$$

where H_m is the m -th triangular Mel filter.

2. **Logarithmic compression:**

$$L_m = \log(E_m + \epsilon)$$

which stabilizes values and mimics loudness perception.

3. DCT-II (decorrelation):

$$c_q = \sum_{m=1}^M L_m \cos \left[\frac{\pi q}{M} \left(m - \frac{1}{2} \right) \right], \quad q = 0, \dots, Q - 1$$

where M is the number of Mel bands and Q the number of coefficients (often $Q = 13$). The c_0 coefficient is sometimes dropped if energy is handled separately.

► Practical notes:

- Libraries like `librosa` or `torchaudio` implement this pipeline directly.
- Typical settings: $M = 40 - 80$ Mel bands, $Q \approx 13$ coefficients.

► Why this works:

- The DCT decorrelates the log-Mel energies.
- Produces compact, roughly uncorrelated features suited for traditional models.
- Historically central to HMM-GMM ASR pipelines.



Automatic Speech Recognition (ASR)

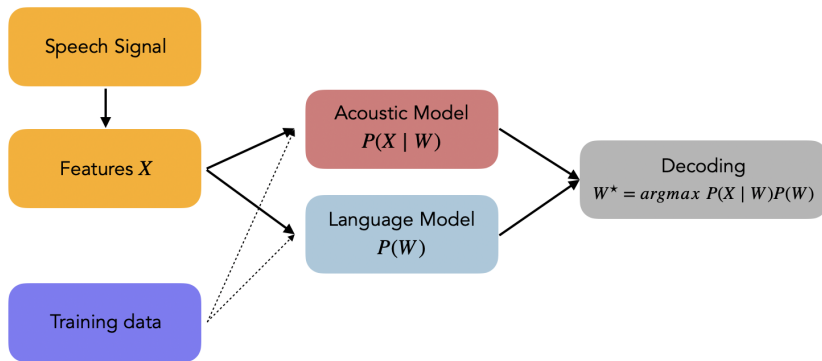
- ▶ Automatic Speech Recognition converts spoken language (audio) into written text.
- ▶ Example:

Audio: "Hey computer, what's the weather?" → Text: "hey computer what's the w

- ▶ How it works (high-level pipeline):
 - Extract features from audio (e.g., log-Mel spectrograms, MFCCs).
 - Pass features through an **acoustic model** to map audio patterns to phonemes or subword units.
 - Use a **language model** to predict the most probable word sequences.
 - Decode the combined outputs into final text.

► Why ASR is important:

- Enables natural language interaction with computers (voice assistants).
- Facilitates transcription, accessibility, and spoken-input NLP applications.



Statistical historical approach to ASR.

- ▶ Traditional ASR systems were built as modular pipelines with interpretable components.
- ▶ Main components:
 - **Acoustic model:** estimates $P(x \mid \text{phones})$, mapping audio features to phonetic units.
 - **Pronunciation dictionary:** converts phones to words.
 - **Language model:** estimates $P(\text{words})$, guiding likely word sequences.
 - **Decoder:** searches for the most probable word sequence given the acoustic and language models.
- ▶ Mathematical formulation:

$$\hat{W} = \arg \max_W P(W \mid X) = \arg \max_W P(X \mid W) P(W)$$

where W = word sequence, X = extracted audio features.

- ▶ Why this approach worked:
 - Easier to engineer and understand.
 - Interpretable: each module can be improved independently.
- ▶ Limitations:
 - Complex, hand-crafted system.
 - Errors can propagate through the pipeline.

- ▶ Modern ASR systems often replace traditional modular pipelines with **neural sequence-to-sequence models**, directly mapping audio features to text.
- ▶ Popular approaches:
 - **CTC (Connectionist Temporal Classification)** – handles variable-length alignments.
 - **Attention-based encoder-decoder** – learns soft alignments between input frames and output tokens.
 - **Transducer models** – integrate CTC-style alignment with recurrent decoding.
- ▶ Advantages:
 - Minimal handcrafting of features or components.
 - Often achieves better accuracy when trained on large datasets.

- ▶ Mathematical intuition (CTC objective):

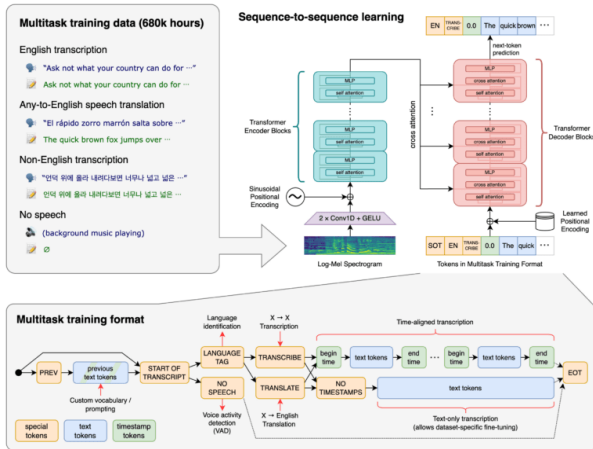
$$P(W | X) = \sum_{\pi \in B^{-1}(W)} \prod_t P(\pi_t | X)$$

where π represents a possible alignment and B is the collapse function that removes blanks and repeats.

- ▶ Reference: <https://distill.pub/2017/ctc/>

- ▶ Whisper is a state-of-the-art, multilingual ASR model developed by OpenAI.
- ▶ Key architecture:
 - Encoder-decoder transformer.
 - Trained on 680,000 hours of labeled and weakly-labeled audio.
 - Takes log-Mel spectrograms as input features.
- ▶ Advantages:
 - Robust to different accents and noisy environments.
 - Supports multitask learning: ASR and optional translation.
- ▶ Example use case:
 - Spoken French: “Je parle français” → can output English translation.

Whisper (OpenAI) (cont.)



Whisper architecture diagram. Image: OpenAI

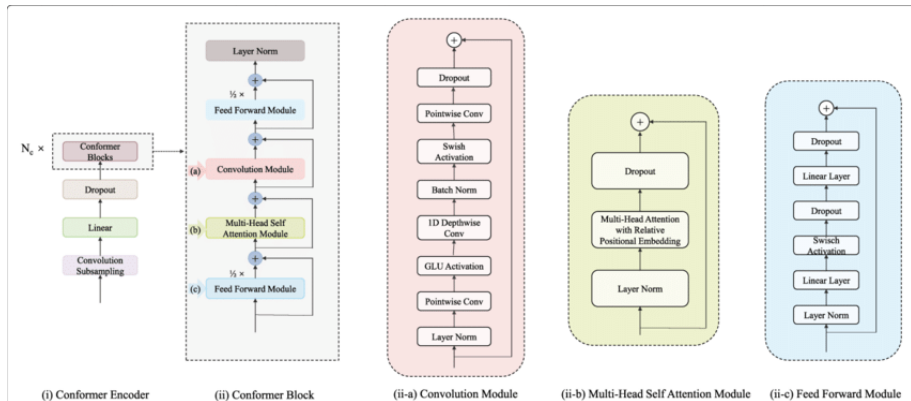
- ▶ Conformer is a hybrid neural model that combines **convolutions** and **transformers** to process speech effectively.
- ▶ Core components:
 - **Convolution:** captures local structures and fine-grained temporal patterns in audio.
 - **Self-attention:** captures global context and long-range dependencies.
- ▶ Achievements:
 - Achieves state-of-the-art performance on LibriSpeech ASR benchmarks.
 - Balances efficiency and accuracy better than pure transformer or CNN models alone.

- Conceptual block equation:

$$y = x + \text{MHSA}(\text{Norm}(x)) + \text{Conv}(\cdot) + \text{FFN}(\cdot)$$

where MHSA = Multi-Head Self-Attention, Conv = convolutional module,
FFN = feed-forward network.

Conformer (Conv + Transformer) (cont.)



Conformer block diagram. Source: Adversarial joint training with self-attention mechanism for robust end-to-end speech recognition

- ▶ Whisper and Conformer represent two modern approaches to ASR with complementary strengths:
- ▶ **Whisper:**
 - Multilingual, trained on massive datasets.
 - Robust to accents, noise, and varied conditions.
 - Open-sourced, general-purpose.
- ▶ **Conformer:**
 - Efficient hybrid model combining convolution + transformer.
 - Excels on English benchmarks like LibriSpeech.
 - Often fine-tuned for specific tasks.

► Trade-offs:

- Whisper is more general-purpose, handles diverse languages and noisy conditions.
- Conformer achieves very low WER ($<2\%$ on LibriSpeech test-clean) but may require task-specific tuning.

- ▶ Understanding the types of errors in ASR helps diagnose model weaknesses and improve performance.
- ▶ Common error types:
 - **Substitution:** a word is replaced by another. Example: “cat” → “cap”
 - **Deletion:** a word is dropped. Example: “the cat sat” → “cat sat”
 - **Insertion:** an extra word is added. Example: “cat sat” → “the cat sat”

- ▶ Word Error Rate (WER) is the most commonly used metric to evaluate ASR performance.
- ▶ It measures the proportion of words that are incorrect, considering substitutions, deletions, and insertions.

- ▶ Formula:

$$\text{WER} = \frac{S + D + I}{N}$$

where: S = substitutions, D = deletions, I = insertions, N = total words in the reference.

- ▶ Example:
 - Reference: "the cat sat on mat"
 - Hypothesis: "the cat sat mat"
 - One deletion $\rightarrow \text{WER} = 1/5 = 20\%$

► Why it matters:

- Standard, intuitive metric for ASR quality.
- Provides a clear measure of accuracy across datasets.

► Limitations:

- WER does not always reflect user satisfaction.
- Some errors may be less critical than others depending on context.

- ▶ Several factors influence how well an ASR system performs in real-world conditions:
- ▶ **Noise level and microphone quality:** Background noise or poor recording devices can degrade feature extraction and recognition accuracy.
- ▶ **Speaker variability:** Differences in accent, pitch, speaking rate, or pronunciation can challenge the acoustic model.
- ▶ **Domain mismatch:** Models trained on casual speech may struggle with specialized domains (e.g., medical or legal speech).
- ▶ **Training data size and diversity:** More diverse and larger datasets generally improve robustness across speakers, accents, and environments.

Spoken Language Understanding (SLU)

- ▶ SLU focuses on understanding the meaning of spoken utterances, going beyond mere transcription.
- ▶ Goal: transform “what was said” → “what was meant.”
- ▶ Input and output:
 - Input: audio signal.
 - Output: structured meaning, typically **intent** + **slots**.
- ▶ Definitions:
 - **Intent**: the action the user wants to perform (e.g., play music, set an alarm).
 - **Slots**: arguments of the intent (e.g., {song: “Yesterday”, artist: “Beatles”}).

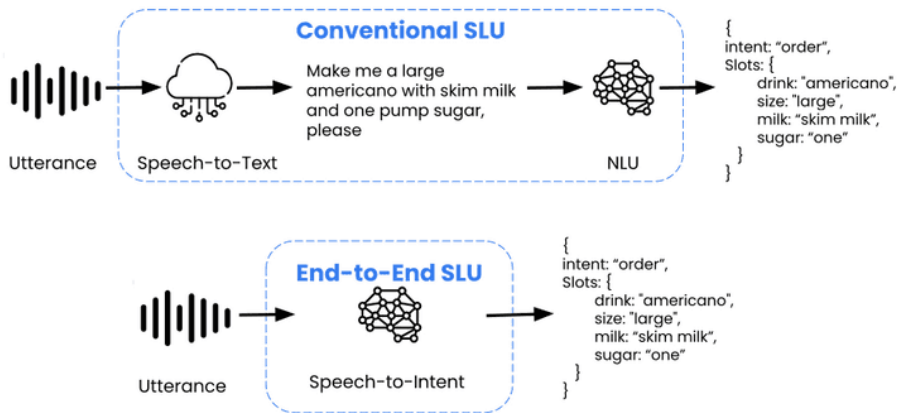
► Example:

- Audio: "Set an alarm for 7 a.m."
- Output: {intent: set_alarm, time: 7:00}

► Why SLU is important:

- Enables interaction with smart assistants and dialogue systems.
- Bridges the gap between raw speech signals and actionable NLP tasks.

Spoken Language Understanding (SLU) (cont.)



Spoken Language Understanding. Source: [PICOVOICE.AI](https://picovoice.ai)

- ▶ Many commercial SLU systems follow a two-step pipeline, separating speech recognition from understanding.
- ▶ Pipeline steps:
 1. **Audio → ASR → Text Transcript** Converts raw audio into written words.
 2. **Text → NLU Model**
 - ▶ **Intent classification:** identifies user's action (e.g., `request_weather`).
 - ▶ **Slot filling:** extracts entities or arguments (e.g., `{location: "London"}`).
 3. **Dialogue manager:** decides system response or action based on intent and slots.

► Advantages of this approach:

- Leverages existing NLP models trained on text.
- Easier to build and maintain using large text corpora.

► Example:

- User says: “What’s the weather like in Paris?”
- ASR output: “what’s the weather like in paris”
- NLU output: {intent: get_weather, location: “Paris”}

- ▶ Direct SLU models bypass transcription and predict meaning directly from audio signals.
- ▶ How it works:
 - Input features: log-Mel spectrograms or wav2vec embeddings.
 - Model architecture: typically CNNs or Transformers for classification or sequence labeling.
 - Output: intent + slots, directly mapped from audio.

► Advantages:

- Avoids error propagation from ASR mistakes.
- Ideal for domain-specific commands where transcription is unnecessary.
- Faster and more efficient than pipeline approaches.

► Example:

- Audio: "Play jazz"
- Direct SLU output: {intent: play_music, genre: "jazz" }
- No intermediate transcript is generated.

- ▶ Spoken Language Understanding faces several real-world challenges:
- ▶ **Error Propagation:** In pipeline systems, ASR mistakes can cause NLU errors. *Example:* “Play rock” misheard as “Play doc” → system fails.
- ▶ **Data Scarcity:** Annotated SLU datasets are often small. Collecting paired (speech, intent, slots) data is expensive.
- ▶ **Multilinguality Code-Switching:** Users often mix languages in the same utterance. *Example:* “Ponme una canción by Shakira.” Systems must handle hybrid language input robustly.
- ▶ **Context Dialogue:** Understanding multi-turn conversations requires memory of previous interactions. *Example:* “Play jazz.” → “Now make it louder.”

- ▶ Spoken Language Understanding powers many real-world applications by extracting meaning from speech, not just transcribing it.
- ▶ Common use cases:
 - **Virtual assistants:** Siri, Alexa, Google Assistant.
 - **Smart homes:** voice-controlled lights, appliances.
 - **Customer service:** routing calls based on intent.
 - **Automotive:** navigation, calls, climate control.
 - **Healthcare:** converting doctors' spoken notes into structured forms.

► Example (Alexa):

- User says: “Turn off the bedroom lights.”
- SLU output: {intent: control_device, device: lights, location: bedroom}

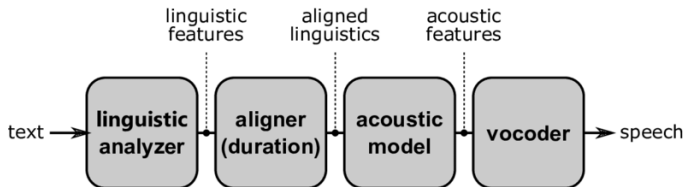
► Why SLU is critical:

- Transcription alone isn't sufficient; systems must act on the meaning of what was said.

Text-to-Speech (TTS) & Vocoders

- ▶ Speech synthesis, also known as Text-to-Speech (TTS), converts written text into spoken audio.
- ▶ How it works:
 - **Input:** raw text.
 - **Processing:** linguistic analysis → acoustic modeling → vocoder.
 - **Output:** waveform (audio file).
- ▶ Why it matters:
 - Accessibility: screen readers for visually impaired users.
 - Voice assistants, audiobooks, GPS navigation, and language learning applications.
- ▶ Example:
 - Text: “Welcome to the lecture” → synthetic speech audio output.

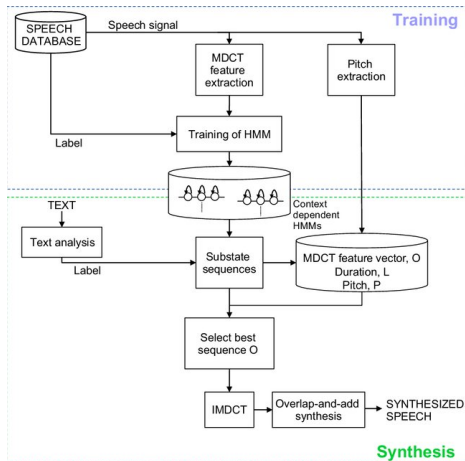
- ▶ Applications: virtual assistants, accessibility, entertainment
- ▶ Key components:
 - Text analysis
 - Acoustic modeling
 - Vocoder synthesis



- ▶ Traditional TTS uses a modular pipeline where each stage handles a specific aspect of text-to-speech conversion.
- ▶ Pipeline steps:
 1. **Text Analysis:** normalize numbers, abbreviations, and punctuation.
 2. **Phonetic Analysis:** convert graphemes to phonemes.
 3. **Prosody Modeling:** predict rhythm, intonation, and stress patterns.
 4. **Acoustic Model:** generate spectrogram-like features representing speech.
 5. **Vocoder:** convert acoustic features into a waveform (audio output).
- ▶ Advantages and limitations:
 - Clear, interpretable stages.
 - Hand-engineered, less flexible than modern neural TTS approaches.

- ▶ Statistical Parametric TTS generates speech by modeling acoustic and prosody parameters statistically, often using Hidden Markov Models (HMMs).
- ▶ How it works:
 - Acoustic and prosody features are predicted via HMMs.
 - Vocoder converts predicted features into a speech waveform.
- ▶ Advantages:
 - Smaller model footprint.
 - Flexible, can adapt to new speakers or styles with limited data.
- ▶ Limitations:
 - Synthesized speech often sounds muffled or buzzy compared to natural human speech.

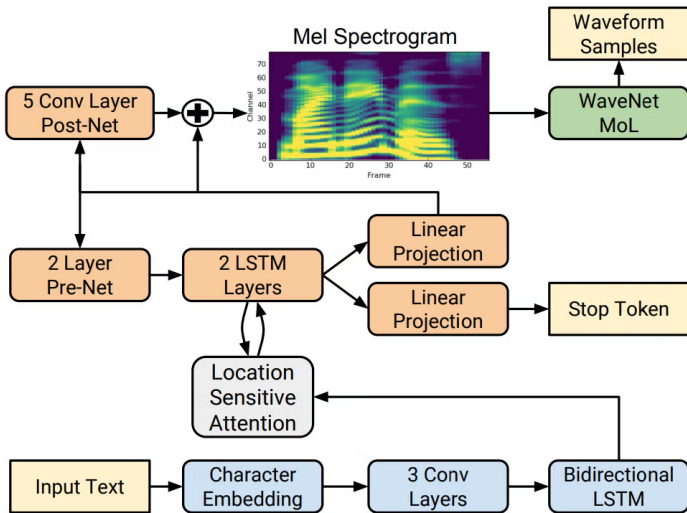
Statistical Parametric TTS (cont.)



HMM-based TTS diagram. Source: [ResearchGate](#)

- ▶ End-to-end TTS system
- ▶ Pipeline:
 - Text $\rightarrow$ Mel-spectrogram (sequence-to-sequence model)
 - Mel-spectrogram $\rightarrow$ WaveNet/GAN vocoder (audio synthesis)
- ▶ Improved prosody and pronunciation
- ▶ Flexible for different voices and languages

Tacotron 2 (cont.)

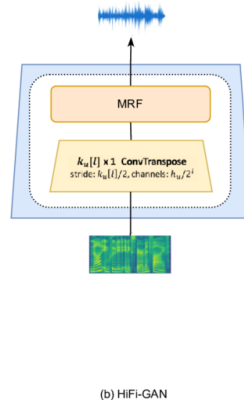
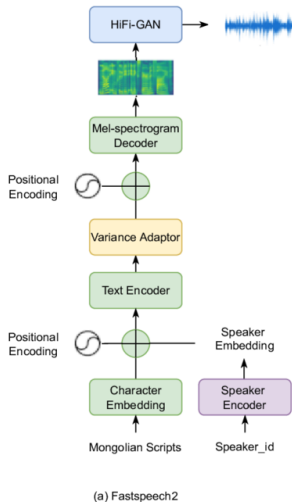




- ▶ Non-autoregressive models: faster inference
- ▶ FastSpeech:
 - Parallel generation of spectrograms
 - Duration prediction for alignment
- ▶ HiFi-GAN:
 - GAN-based vocoder
 - High-fidelity, real-time synthesis



- ▶ Glow-TTS:
 - Flow-based generative model
 - Efficient and expressive speech synthesis
- ▶ Diffusion models:
 - Iterative refinement of audio
 - State-of-the-art naturalness



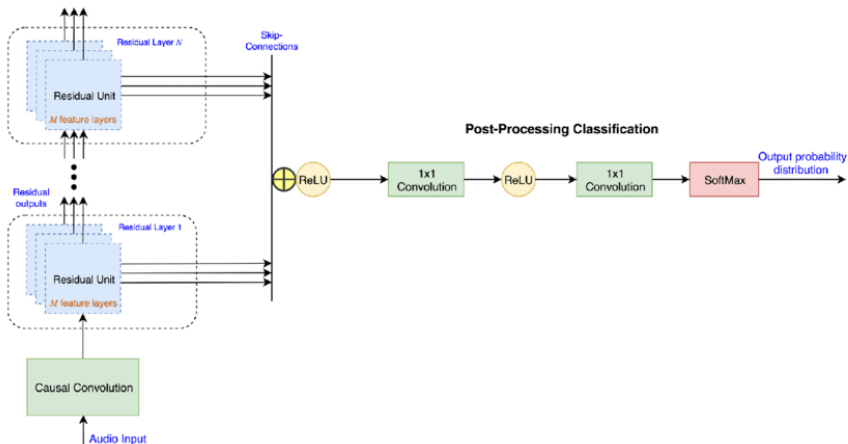
- ▶ Neural vocoders convert intermediate representations (e.g., spectrograms) into waveforms, producing high-quality speech.
- ▶ Popular approaches:
 - **WaveNet (2016)**: autoregressive model, produces very natural speech but relatively slow.
 - **WaveGlow, Parallel WaveGAN, HiFi-GAN**: faster, real-time neural vocoders.
- ▶ Core idea (WaveNet):

$$p(\mathbf{x}) = \prod_t p(x_t \mid x_{<t})$$

Each audio sample is modeled conditionally on all previous samples.

- ▶ Why it matters:
 - The quality of the vocoder directly impacts the perceived naturalness of synthesized speech.

Neural Vocoder (cont.)



WaveNet architecture. Source: DeepMind

- ▶ Measuring TTS quality requires both human judgment and objective metrics.
- ▶ Common evaluation methods:
 - **MOS (Mean Opinion Score):** human listeners rate naturalness on a 1–5 scale.
 - **AB preference tests:** humans choose their preferred sample between two systems.
 - **Objective metrics:**
 - ▶ MCD (Mel-Cepstral Distortion)
 - ▶ PESQ (Perceptual Evaluation of Speech Quality)
- ▶ Why evaluation matters:
 - Human perception is the ultimate measure of speech quality.
 - Automated metrics help accelerate development and iteration.

- ▶ Modern TTS systems face several practical and ethical challenges:
- ▶ **Prosody Emotion:** Generating natural intonation, emphasis, and expressive speech remains difficult.
- ▶ **Cross-Speaker Synthesis:** Voice cloning or multi-speaker synthesis requires adapting models to new voices without losing quality.
- ▶ **Low-Resource Languages:** Limited data makes it challenging to build high-quality TTS systems for many languages.
- ▶ **Ethical Issues:** Risk of misuse in deepfakes or generating fake celebrity voices.

Evaluation Metrics

- ▶ Word Error Rate (WER) is a standard metric to evaluate ASR transcription accuracy. It is based on the edit distance between the reference and the hypothesis.
- ▶ Formula:

$$\text{WER} = \frac{S + D + I}{N}$$

where:

- S = substitutions
- D = deletions
- I = insertions
- N = total words in reference

► Example:

- Reference: “the cat sat on the mat”
- Hypothesis: “the cat sat mat”
- Errors: 1 deletion $\rightarrow$ $WER = 1/6$ 16.7%

► Why it matters:

- Provides an intuitive, widely-used measure of transcription accuracy for ASR systems.

- ▶ Perplexity (PPL) measures how well a language model predicts a sequence of words. Intuitively, it reflects “on average, how many choices the model is unsure about” per word.
- ▶ Formula:

$$\text{PPL}(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i) \right)$$

- ▶ How to interpret:
 - Lower perplexity → model assigns higher probability to the test set sequences → better predictions.

- ▶ Why it matters:
 - Standard metric for evaluating language models in ASR and NLG.
- ▶ Example:
 - Bigram model on a test set $\rightarrow$ PPL = 200 (poor).
 - Transformer LM $\rightarrow$ PPL = 20 (much better).

- ▶ BLEU is an n-gram overlap metric commonly used to evaluate text generation tasks such as machine translation and ASR outputs. It compares generated text against reference text using precision of n-grams.
- ▶ Formula:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

where:

- p_n = precision of n-grams
- w_n = weight for each n-gram order (usually uniform)
- BP = brevity penalty to penalize short hypotheses

► How it works:

- BLEU evaluates how many n-grams in the hypothesis match the reference.
- Missing or misordered words reduce the score.

► Example:

- Reference: “the cat is on the mat”
- Hypothesis: “the cat on mat”
- Unigram precision = $4/4 = 1.0$
- Bigram precision = $2/3 \approx 0.67$
- BLEU penalizes missing words via BP.

► Why it matters:

- Easy to compute and widely adopted for automatic evaluation of generated text.

- ▶ ROUGE is a recall-focused metric widely used in text summarization to evaluate how much of the reference content is captured by the generated text.
- ▶ Common variants:
 - **ROUGE-N**: recall of n-grams.
 - **ROUGE-L**: based on the longest common subsequence.
- ▶ Formula (ROUGE-N):

$$\text{ROUGE-N} = \frac{\sum_{n\text{-gram} \in \text{Ref}} \min(\text{count}_{\text{hyp}}, \text{count}_{\text{ref}})}{\sum_{n\text{-gram} \in \text{Ref}} \text{count}_{\text{ref}}}$$

► Example:

- Reference: “the cat sat on the mat”
- Hypothesis: “the cat sat on mat”
- ROUGE-1 recall = $5/6$ 83%

► Why it matters:

- Good for checking content coverage in summarization tasks.

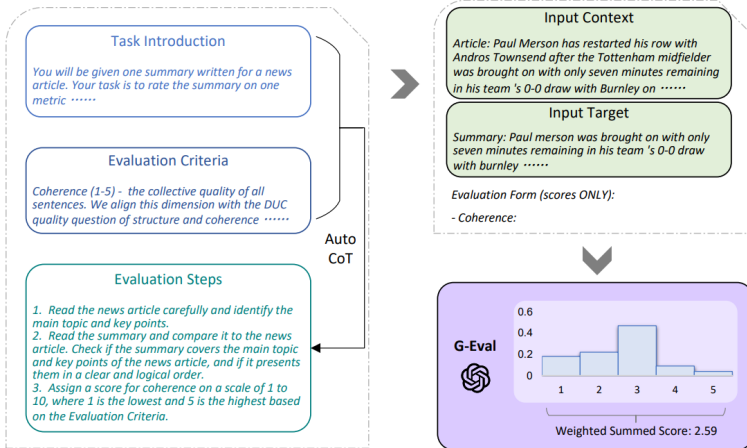
- ▶ **METEOR:** Measures text generation quality considering not just exact word matches but also stemming, synonyms, and paraphrases. Tends to correlate better with human judgment than BLEU.
- ▶ **TER (Translation Edit Rate):** Counts the minimum number of edits—insertions, deletions, substitutions, and shifts—needed to match the reference text. Lower TER indicates better performance.
- ▶ Why these metrics matter:
 - Often used alongside BLEU and ROUGE to get a more complete evaluation of generated text quality.

- ▶ **Automated Metrics** (e.g., WER, BLEU, ROUGE):
 - **Postive:** Cheap, fast, and reproducible.
 - **Negative:** Limited depth; may not capture fluency, naturalness, or true meaning.
- ▶ **Human Evaluation:**
 - **Postive:** Captures fluency, naturalness, and semantic meaning.
 - **Negative:** Expensive, slow, and less reproducible.
- ▶ **Why both matter:**
 - Automated metrics allow rapid iteration and benchmarking.
 - Human evaluation ensures quality aligns with real-world perception.
 - Best practice: combine both depending on the task.

- ▶ Reference: “The cat sat on the mat.”
- ▶ Hypotheses:
 - Hypothesis A: “The cat is on the mat.”
 - Hypothesis B: “A feline rested on the rug.”
- ▶ Metric evaluation:
 - **BLEU:** $A \not\sim B$ (closer word overlap with reference)
 - **ROUGE:** $A \not\sim B$ (better recall of reference n-grams)
 - **LLM/semantic judge:** $B \sim A$ (both semantically correct)
- ▶ Takeaway:
 - Different metrics capture different aspects of quality.
 - Word overlap metrics may disagree with human or semantic judgment.

LLM Judges

- ▶ Large Language Models (LLMs) like GPT, LLaMA, and Claude can serve as automatic evaluators, replacing or augmenting human judges for tasks such as translation, summarization, or dialogue evaluation.
- ▶ How they work:
 - Provide the LLM with a reference text (gold standard) and a candidate output (model prediction).
 - Supply evaluation instructions or prompts to guide judgment.
- ▶ Why they are useful:
 - LLMs understand meaning and style, not just surface-level word overlap.
 - They offer evaluations that are closer to human judgment compared to metrics like BLEU or ROUGE.



NLG Evaluation using GPT-4 with Better Human Alignment. Source: [G-Eval](#)

- ▶ LLMs can automatically evaluate model outputs by comparing them against references using semantic understanding.
- ▶ Core mechanism:
 - **Prompt design:** Ask the LLM to rate fluency, coherence, and adequacy.
Example prompt: “On a scale of 1–5, how well does Candidate match Reference?”
 - **Scoring:** Produce a numeric score or comparative ranking.
 - **Calibration:** Aggregate multiple runs to reduce bias and improve consistency.

- Conceptual equation:

$$\text{Score}(C, R) = f_{\theta}(\text{prompt}, C, R)$$

where f_{θ} is the LLM evaluating candidate C against reference R .

- Techniques:

- **Pointwise:** rate a single candidate.
- **Pairwise:** compare two candidates, pick the better one.
- **Listwise:** rank multiple candidates.

- ▶ Motivation: Traditional metrics and human evaluation have limitations.
 - **N-gram metrics (BLEU/ROUGE):**
 - Miss paraphrases
 - Ignore fluency and coherence
 - **Human evaluation:**
 - Reliable
 - Expensive and slow
- ▶ LLM Judges provide a middle ground:
 - Understand semantic meaning
 - Cheap and scalable
 - Flexible across tasks: machine translation, summarization, ASR post-processing

► Example:

- Reference: "The boy is playing soccer."
- Hypothesis A: "A child plays football."
- Hypothesis B: "The girl eats pizza."
- BLEU punishes Hypothesis A (word mismatch), but LLM Judge favors A as semantically correct.

- ▶ LLM Judges can be applied across multiple NLP and speech tasks:
 - **Machine Translation:** Evaluate adequacy and fluency better than BLEU.
 - **Summarization:** Assess relevance and detect hallucinations.
 - **Dialogue Systems:** Rank chatbot responses (e.g., MT-Bench, Chatbot Arena).
 - **ASR Post-processing:** Rate transcript quality against reference.
 - **TTS/NLG Evaluation:** Measure expressiveness, naturalness, and style.
- ▶ Example (MT-Bench): GPT-4 performs pairwise comparisons of chatbot responses to select the better reply.

- ▶ While LLM Judges offer semantic evaluation, they have notable limitations:
 - **Biases:**
 - ▶ Position bias: may favor the first candidate.
 - ▶ Verbosity bias: longer answers can be preferred.
 - ▶ Self-enhancement: may favor outputs similar to its own style.
 - **Stability:** Scores can vary with minor changes in prompt wording.
 - **Reproducibility:** LLM outputs are not always deterministic.
 - **Transparency:** Evaluation reasoning is often a black-box, making it hard to interpret scores.

- ▶ To maximize reliability and minimize bias when using LLM Judges:
 - Use **pairwise comparisons** instead of pointwise scoring to reduce bias.
 - **Randomize candidate order** to avoid position bias.
 - **Calibrate** with a small set of human ratings to align scoring.
 - Report **inter-rater reliability** between LLM and human evaluators.
 - **Combine** LLM judgments with traditional metrics (BLEU, ROUGE, WER) for a holistic evaluation.
- ▶ Why this matters:
 - Ensures evaluations are trustworthy, reproducible, and meaningful.

Challenges & Current Research



- ▶ **Robustness:** Handling noise, accents, and low-quality microphones.
- ▶ **Multilinguality & Code-Switching:** Supporting diverse populations and hybrid language input.
- ▶ **Low-Resource Speech:** Many languages lack sufficient training data for ASR/TTS models.
- ▶ **Expressive Speech Synthesis:** Capturing prosody, emotion, and enabling style transfer.
- ▶ **Evaluation:** Moving beyond BLEU/WER to semantic, human-centered, and LLM-based metrics.
- ▶ **Ethics:** Addressing risks like deepfake voices and protecting privacy in speech datasets.

- ▶ Speech is a rich modality for natural language processing, bridging sound and meaning.
- ▶ Audio features such as **MFCC**, **Mel**, and **Log-Mel** form the foundation for ASR and TTS systems.
- ▶ The spoken interaction cycle involves **ASR** $\rightarrow$ **SLU** $\rightarrow$ **TTS**, enabling complete end-to-end communication.
- ▶ **Evaluation** is crucial: combine automated metrics (WER, BLEU, ROUGE) with human and LLM-based assessments.
- ▶ Future directions:
 - More **end-to-end models** for robust and expressive speech understanding/synthesis.
 - Improved, **semantic and human-centered evaluation metrics**.
 - Stronger **ethical safeguards** against misuse.



References

- [1] Jurafsky, D., & Martin, J. H. (2023). *Speech & Language Processing (3rd edition draft)*. <https://web.stanford.edu/~jurafsky/slp3/>
- [2] Rabiner, L., & Juang, B.-H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.
- [3] Deng, L., & Li, X. (2013). *Deep Learning for Speech and Language*. Springer.
- [4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [5] Rabiner, L. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.
- [6] Graves, A., Fernandez, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *ICML*.
- [7] Graves, A. (2012). Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*.

- [8] Gulati, A., Qin, J., Chiu, C.-C., et al. (2020). Conformer: Convolution-augmented transformer for speech recognition. *Interspeech*.
- [9] Radford, A., et al. (2022). Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*.
- [10] Wang, Y., et al. (2017). Tacotron: Towards end-to-end speech synthesis. *arXiv preprint arXiv:1703.10135*.
- [11] Shen, J., et al. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. *ICASSP*.
- [12] Ren, Y., et al. (2019). FastSpeech: Fast, robust and controllable text to speech. *arXiv preprint arXiv:1905.09263*.
- [13] Kim, J., et al. (2021). VITS: Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. *arXiv preprint arXiv:2106.06103*.
- [14] van den Oord, A., et al. (2016). WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.

- [15] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a method for automatic evaluation of machine translation. *ACL*.
- [16] Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. *ACL Workshop on Text Summarization*.
- [17] Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *ACL Workshop on Intrinsic and Extrinsic Evaluation Measures*.
- [18] Snover, M., et al. (2006). A study of translation edit rate with targeted human annotation. *Association for Machine Translation in the Americas (AMTA)*.
- [19] Liu, P., et al. (2023). G-Eval: Using large language models as automatic judges. *arXiv preprint arXiv:2303.12712*.
- [20] Zheng, X., et al. (2023). Judging LLM-as-a-Judge. *arXiv preprint arXiv:2305.10565*.
- [21] Chiang, C., et al. (2023). MT-Bench and Chatbot Arena: Evaluating LLMs with pairwise comparison. *arXiv preprint arXiv:2304.XXXXX*.

- [22] Panayotov, V., Chen, G., Povey, D., & Khudanpur, S. (2015). Librispeech: An ASR corpus based on public domain audio books. *ICASSP*.
- [23] Ardila, R., et al. (2019). CommonVoice: A massively-multilingual speech corpus. *arXiv preprint arXiv:1912.06670*.
- [24] Hernandez, F., et al. (2018). TED-LIUM 3: Speech recognition corpus. *LREC*.
- [25] Carletta, J., et al. (2005). The AMI meeting corpus: A pre-announcement. *Language Resources and Evaluation*.
- [26] LJSpeech dataset. <https://keithito.com/LJ-Speech-Dataset/>
- [27] VCTK Corpus. <https://datashare.ed.ac.uk/handle/10283/3443>
- [28] HiFi-TTS dataset. <https://github.com/NVIDIA/NeMo>
- [29] ATIS dataset.
<https://www.kaggle.com/datasets/siddhadev/atis-intent-slot-dataset>

- [30] SLURP dataset. <https://github.com/pswietojanski/slurp>
- [31] Whisper ASR Demo. <https://huggingface.co/spaces/openai/whisper>
- [32] Tacotron2 + WaveGlow Demo. https://colab.research.google.com/github/NVIDIA/DeepLearningExamples/blob/master/PyTorch/SpeechSynthesis/Tacotron2/notebooks/Tacotron2_WaveGlow_Inference.ipynb
- [33] Google TTS API demo. <https://cloud.google.com/text-to-speech>
- [34] Coqui TTS. <https://coqui.ai>
- [35] Li, X., et al. (2020). Recent advances in end-to-end ASR. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*.
- [36] Qian, Y., et al. (2021). Recent advances in spoken language understanding. *Speech Communication*.
- [37] Tan, T., et al. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2105.XXX*.

- [38] Chiang, C., et al. (2023). LLM-as-a-Judge: Survey. *arXiv preprint arXiv:2304.XXXXX*